

Lab 15 - Expected output reference

UIDs, resourceVersions, timestamps and the exact number of seconds the volume refresh takes are runtime values and will differ. The base64 strings below are deterministic, because they encode the fixed placeholder strings in data/dummy-credentials.env.

Every credential-shaped value in this lab is a placeholder beginning with DUMMY-.

checks.sh enforces that with a dedicated assertion.

bash verification/checks.sh - clean run

```
== Lab 15 verification: Secrets, ConfigMaps and safe injection ==
PASS namespace kcna-lab15 exists
PASS ConfigMap booking-app-config app-settings.json matches data/app-settings.json byte-for-byte
PASS ConfigMap booking-app-config has all 4 keys from data/feature-flags.env with matching values
PASS ConfigMap booking-app-config-v1 is immutable
PASS Secret booking-credentials has all 3 keys from data/dummy-credentials.env
PASS Secret booking-credentials values base64-decode to the data/dummy-credentials.env values
PASS Secret booking-credentials contains only obvious DUMMY placeholders
PASS Secret booking-api-example stores stringData as base64 under .data and returns no stringData
PASS env-consumer resolved GREETING from a configMapKeyRef
PASS env-consumer resolved DB_USERNAME from a secretKeyRef
PASS env-consumer bulk-imported the FEATURE_* keys with the CFG_ prefix
PASS env-consumer omitted the optional key that does not exist
PASS volume-consumer projected only the 2 requested ConfigMap keys, with greeting.txt renamed
PASS volume-consumer secret file mode is 0440 with group 1000 (fsGroup)
PASS volume-consumer secret volume is tmpfs (never written to node disk)
PASS volume-consumer subPath mount exists alongside the directory mount
PASS RECORDED: this cluster has NO --encryption-provider-config (Secrets are plaintext in etcd)
-- 17 passed, 0 failed --
```

Exit status 0.

Run this after restoring the ConfigMap at the end of Step 8 and after deleting

broken-key-consumer at the end of section 6. If you run it mid-Step-8 the

app-settings.json comparison will correctly fail, because you deliberately patched

the ConfigMap to log_level: debug.

The last line is a record, not a pass/fail judgement. On a cluster that *does* configure encryption at rest it reads:

```
PASS RECORDED: this cluster DOES set --encryption-provider-config (Secrets are encrypted at rest)
```

Key intermediate outputs

The Secret as stored (Step 6)

```
apiVersion: v1
data:
  BOOKING_API_TOKEN: RFVNTVktbm90LWEtcVhbc10b2t1bi0wMDAwMDAwMDAw
  BOOKING_DB_PASSWORD: RFVNTVktbm90LWEtcVhbc1wYXNzd29yZC1jaGFuZ2UtbWU=
  BOOKING_DB_USERNAME: RFVNTVktbm90LWEtcVhbc11c2VybmFtZQ==
kind: Secret
type: Opaque
```

The one-line disproof of "Kubernetes encrypts secrets"

```
$ kubectl get secret booking-credentials -n kcna-lab15 -o jsonpath='{.data.
BOOKING_DB_PASSWORD}' | base64 -d
DUMMY-not-a-real-password-change-me
```

No key, no password, no privilege beyond get secrets. Base64 is an encoding.

The describe view masks the values:

```
Data
====
BOOKING_API_TOKEN:      33 bytes
BOOKING_DB_PASSWORD:   35 bytes
BOOKING_DB_USERNAME:   25 bytes
```

...while describe configmap prints its values in full. Neither behaviour is a security control; -o yaml ignores both.

And the state of this cluster:

```
$ kubectl get pod -n kube-system -l component=kube-apiserver \
  -o jsonpath='{.items[0].spec.containers[0].command}' | tr ',' '\n' | grep -i encryption-
provider \
  || echo "NOT SET -> Secrets are stored in etcd in plaintext on this cluster"
NOT SET -> Secrets are stored in etcd in plaintext on this cluster
```

Environment injection (Step 7)

```
CFG_APP_GREETING=Meridian Freight booking API
CFG_FEATURE_BULK_UPLOAD=true
CFG_FEATURE_LEGACY EDI=false
CFG_FEATURE_LIVE_VESSEL_ETA=false
CFG_FEATURE_MULTILEG_QUOTES=true
DB_USERNAME=DUMMY-not-a-real-username
GREETING=Meridian Freight booking API
MULTILEG_ENABLED=true
```

Absent on purpose, and both are teaching points:

Missing variable	Why
OPTIONAL_TUNING	Its configMapKeyRef sets optional: true and the key does not exist. Without optional, this is CreateContainerConfigError

Missing variable	Why
CFG_app-settings.json	Not a valid shell identifier, so envFrom skipped it. kubelet records a Warning InvalidEnvironmentVariableNames event and the container starts anyway

Leakage demonstration:

```
$ kubectl exec -n kcna-lab15 env-consumer -- sh -c 'tr "\0" "\n" < /proc/1/environ | grep DB_USERNAME'
DB_USERNAME=DUMMY-not-a-real-username
```

Volume injection (Step 8)

```
/etc/booking/config:
lrwxrwxrwx app-settings.json -> ../data/app-settings.json
lrwxrwxrwx greeting.txt -> ../data/greeting.txt
```

Two files, not seven - items: projected only the requested keys and renamed APP_GREETING to greeting.txt.

```
$ kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'ls -lL /etc/booking/credentials/BOOKING_DB_PASSWORD; id'
-r--r----- 1 root 1000          35 Sep  5 10:40 /etc/booking/credentials/BOOKING_DB_PASSWORD
uid=1000 gid=1000 groups=1000
```

Mode 0440, group 1000 - set by fsGroup: 1000. Without fsGroup this file is root:root and the non-root container gets Permission denied.

```
$ kubectl exec -n kcna-lab15 volume-consumer -- sh -c 'mount | grep booking/credentials'
tmpfs on /etc/booking/credentials type tmpfs (ro,relatime,...)
```

The subPath result - the headline of Step 8

After patching the ConfigMap and waiting for kubelet's sync:

```
/etc/booking/config/app-settings.json: "log_level": "debug",
/etc/booking/pinned-settings.json: "log_level": "info",
```

Same ConfigMap, same Pod, same container. The directory mount updated; the subPath mount did not and never will. Environment variables did not update either.

Failure-injection reference (Section 6)

```
$ kubectl apply -f manifests/50-pod-broken-key.yaml
pod/broken-key-consumer created

$ kubectl get pod broken-key-consumer -n kcna-lab15
NAME                READY   STATUS              RESTARTS   AGE
broken-key-consumer  0/1     CreateContainerConfigError  0          12s
```

The decisive event line:

```
Warning Failed 10s (x3 over 24s) kubelet Error: couldn't find key APP_TIER in ConfigMap
kcna-lab15/booking-app-config
```

Static validation cannot see this:

```
$ kubeconform -strict -summary manifests/50-pod-broken-key.yaml
Summary: 1 resource found parsing 1 file - Valid: 1, Invalid: 0, Errors: 0, Skipped: 0
```

Immutability rejections:

```
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid: data: Field
is immutable
Error from server: failed to patch: ConfigMap "booking-app-config-v1" is invalid: immutable:
Field is immutable
```

Environment caveats recorded for this lab

Caveat	Observed effect	Why it matters	
Secrets are base64-encoded, not encrypted	`kubectl get secret -o jsonpath \	base64 -d` returns plaintext with no key	The lab demonstrates this rather than asserting it. Mitigations are RBAC, encryption at rest and an external store
This kind cluster sets no --encryption-provider-config	Secret values are stored in etcd in plaintext	Verified directly against the kube-apiserver Pod's command line and recorded by checks.sh	
Volume refresh is not instant	A ConfigMap change reaches a directory mount in roughly one kubelet sync period - commonly 60-90 s, occasionally longer	Not a hang. The README polls rather than asserting a fixed delay	
subPath mounts never update	The pinned file keeps its Step-2 content for the container's whole life	Bind mount made at container creation; kubelet cannot re-point it without a restart	
Environment variables never update	env-consumer keeps the original values after the ConfigMap changes	Frozen at process start. Roll the workload to change them	
A 0440 Secret needs fsGroup	Without it, a non-root container gets Permission denied on its own credential	Volume files are owned by root; fsGroup sets the group	
macOS base64	Older releases reject -d and need -D	checks.sh probes for the working flag at runtime	
No real credentials anywhere	Every value starts with DUMMY-	Enforced by a dedicated check that fails if a non-DUMMY- value is ever introduced	